

1. Introduction

Efficient real-time navigation is a critical component of modern intelligent transportation systems, yet the dynamic nature of urban environments presents significant challenges to traditional pathfinding algorithms. While heuristic search algorithms like A* and its bidirectional variant have proven effective in static environments, they often fall short in complex urban settings characterized by dynamic, unpredictable events such as traffic congestion, accidents, and road closures. These events render pre-calculated shortest paths obsolete and necessitate algorithms that can adapt to real-time information.

The primary challenge arises from the fact that real-world urban pathfinding is not a static problem. The optimal path is rarely the shortest geometric one, as it is constantly influenced by dynamic variables such as traffic congestion and unexpected incidents like accidents or road closures.

2. Related Work

2.1. Foundational Pathfinding Algorithms and Their Limitations

traditional pathfinding approaches, often rooted in algorithms like Dijkstra's or A*, provide efficient solutions for optimal routes in static environments. The Bidirectional A* (BA*) algorithm, a widely adopted variant, further enhances efficiency by searching simultaneously from both start and goal nodes. However, despite its advantages, BA* faces inherent challenges. As detailed by Whangbo (2007), modifications are often necessary to ensure optimality guarantees while maintaining efficiency. Furthermore, recent work by Nannicini et al. (2012) has explored the adaptation of Bidirectional A* for time-dependent road networks, highlighting the challenges and potential solutions for ensuring efficiency and optimality in such dynamic contexts. A systematic literature review by Foad et al. (2021) highlights that classic A* and its variants, including BA*, tend to struggle with performance degradation on larger grids, underscoring the limitations of these foundational algorithms when applied to the vast and complex networks found in urban environments. More advanced enhancements, such as segmented evaluation functions with dynamic weights and steering cost integration, have also been proposed to

address issues like excessive node traversal and redundant turning points in specific environments (Li et al., 2023). Furthermore, the concept of Multi-Heuristic A* (MHA*), which leverages multiple heuristics to improve search efficiency and handle complex environments, offers another avenue for enhancing A*'s performance in challenging scenarios (Aine et al., 2015). This performance bottleneck necessitates further enhancements to meet modern real-time navigation demands.

2.2. Dynamic Traffic-Aware Routing in Urban Environments

The dynamic nature of urban traffic and unpredictable events render static shortest paths insufficient. Research has extensively explored methods for dynamic traffic-aware routing. **Fleischmann et al. (2004)** demonstrated significant performance improvements in dynamic vehicle routing by integrating online traffic information to update travel times. Similarly, **Jerbi et al. (2006)** introduced the GyTAR protocol for Vehicular Ad Hoc Networks (VANETs), which utilizes real-time traffic density as a primary factor in guiding routing decisions in urban settings. More recently, **Wan et al. (2019)** showcased systems that leverage mobile crowd-sourced data to model road status (e.g., blocked or open) and real-time traffic speeds, thereby providing adaptive optimal routes. Furthermore, algorithms like SALA (Self-Adaptive Learning Algorithm) have demonstrated how individual vehicle agents can learn and adapt their route guidance in dynamic urban traffic, considering uncertainties and multiple cost factors (Yan et al., 2018). Building on earlier foundational work in Dynamic Traffic Assignment (DTA) that emphasized anticipating congestion through projected link volumes and dynamic link impedance functions (Janson, 1991), these studies collectively validate the critical importance of incorporating real-time dynamic information to adapt path costs and guide navigation effectively in ever-changing urban landscapes.

2.3. Penalty Functions and Constraint Handling

A robust approach to manage dynamic environmental constraints in pathfinding involves the use of penalty functions, where undesirable paths or areas incur an increased cost. **Avella et al. (2002)** provided a theoretical foundation for this concept, demonstrating how penalty function heuristics can effectively handle resource-constrained shortest path problems by transforming hard constraints into adaptable cost increases. A practical

application is seen in the work of **Hu et al. (2016)** , who developed a "translation module" to convert time delays caused by traffic lights into an equivalent distance penalty, which is then integrated into an A* algorithm's heuristic. This approach demonstrates the viability of modeling specific environmental challenges as quantifiable costs. Furthermore, the work by **Sud et al. (2008)** on adaptive roadmaps in crowd simulation incorporates cost functions that explicitly penalize narrow passages and crowded regions, treating them as "soft obstacles" to be avoided. Beyond direct penalty mechanisms, meta-heuristic approaches have also been refined to efficiently navigate highly constrained search spaces, offering strategies to avoid local optima and enforce complex rules (Garvin et al., 2011). This body of work underscores that modeling environmental restrictions as penalties is an effective strategy for guiding pathfinding algorithms.

2.4. Hybrid Global/Local Planning Architectures

The complexity of dynamic urban environments often necessitates a multi-layered approach to pathfinding, combining global strategic planning with local reactive avoidance. This has led to the development of hybrid architectures. For instance, **Zhang et al. (2025)** proposed an integrated system that uses an improved A* for global path generation, whose waypoints then guide a Dynamic Window Algorithm (DWA) for local, real-time dynamic obstacle avoidance. Similarly, **Liu et al. (2022)** combined an optimized A* algorithm with an Artificial Potential Field (APF) method, where A* provides the global route and APF handles local obstacle avoidance and path smoothing. These hybrid models highlight the practical benefits of separating concerns: a global planner ensures overall route optimality or feasibility, while a local planner handles immediate, unforeseen changes and provides smooth trajectory execution.

2.5. Summary of Literature and Research Gap

The existing literature offers valuable insights into addressing dynamic urban navigation challenges. Foundational algorithms like A* and Bidirectional A* are essential starting points but require significant enhancements for large, dynamic networks. Approaches like dynamic traffic assignment and crowd-sourced routing effectively leverage real-time data to inform path costs. The concept of penalty functions is well-established for integrating

constraints into pathfinding costs. Furthermore, hybrid global/local architectures represent a mature strategy for combining strategic planning with reactive avoidance.

However, a critical research gap persists in the realm of global path planners designed for explicit obstacle avoidance. While methods exist to adjust costs based on dynamic traffic or to perform local reactive avoidance, there is a distinct lack of research focusing on enhancing _global pathfinding algorithms_ with a _configurable and explicitly quantifiable_ mechanism for avoiding discrete, dynamic obstacles. Specifically, current literature does not adequately provide a systematic analysis of the inherent trade-offs between achieving a desired level of safety (i.e., obstacle clearance), maintaining path efficiency (length), ensuring pathfinding reliability (failure rate), and managing computational cost within a unified, tunable framework. This approach of analyzing competing objectives aligns conceptually with studies on performance trade-offs in other domains, such as the diversity-multiplexing tradeoff in bidirectional communication protocols (Liu & Kim, 2010).

This paper addresses this gap by proposing and evaluating an enhanced Bidirectional A* algorithm that incorporates a spatially-indexed, hard exclusion mechanism with a configurable avoidance radius. Our primary contribution is not just the algorithm itself, but the comprehensive, quantitative analysis of its performance across the competing objectives of safety, efficiency, and reliability. This work provides a practical framework for building tunable, risk-aware navigation systems that can adapt to varying real-world conditions and user preferences.

2.6. Paper Outline

The remainder of this paper is structured as follows: Section 3 details the methodology, including the road network representation, the core pathfinding algorithm, and the obstacle avoidance mechanism. Section 4 presents the experimental setup. Section 5 provides a detailed analysis of the results. Finally, Section 6 discusses the implications of the findings and outlines directions for future work.

3. Methodology

3.1. Road Network Representation

The foundation of our real-time urban navigation system is a robust and accurate representation of the underlying road network. For this research, we utilize OpenStreetMap (OSM) data, a globally crowdsourced geospatial database renowned for its detailed and up-to-date urban infrastructure information. The specific geographical area of focus for our experimental validation encompasses the twin cities of Kathmandu and Lalitpur, Nepal, chosen due to their complex and dynamic urban road characteristics, which include varied road types, intricate intersections, and frequent real-world obstacles.

The process begins with the extraction of the road network using the `osmnx`` Python library. `osmnx`` efficiently downloads street network data from OSM and constructs it into a `networkx`` graph object, a powerful Python package for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks. Each intersection within the road network is represented as a **node** in our graph, uniquely identified by its OSM ID and associated with precise latitude and longitude coordinates. Conversely, the road segments connecting these intersections are modeled as **edges**. Each edge is characterized by attributes crucial for pathfinding, including its length (in meters), the road type, and other relevant metadata derived from OSM.

A critical preprocessing step involves simplifying and cleaning the raw `networkx`` graph to optimize it for pathfinding queries. This includes:

- **Projection to a Unified Coordinate System:** All geographical coordinates are projected to a local Universal Transverse Mercator (UTM) system. This conversion is essential for accurate distance calculations and spatial indexing in a planar coordinate system, minimizing distortions inherent in latitude/longitude-based measurements, especially over smaller urban areas.
- **Graph Simplification:** Complex intersections or redundant nodes (e.g., nodes that merely represent a bend in a road rather than an actual junction) are consolidated to reduce the graph's overall size and improve query performance without sacrificing topological accuracy.

- **Edge Weight Assignment:** The primary weight assigned to each edge is its physical length, derived from the projected coordinates. This serves as the base cost for traversing a road segment.
- **Serialization for Performance:** The pre-processed graph is serialized and saved to disk in `gpickle` format. This allows for rapid loading of the graph into memory upon server startup, significantly reducing initialization time and ensuring that the pathfinding service can become operational almost instantaneously.

This comprehensive approach to road network representation ensures that our pathfinding algorithm operates on a topologically accurate, computationally efficient, and spatially coherent graph structure, forming a solid basis for real-time navigation and obstacle avoidance.

3.2. Core Pathfinding Algorithm

The core of our proposed solution is a customized **Bidirectional A*** search algorithm, which has been enhanced with a spatial index to allow for efficient, real-time obstacle avoidance. The algorithm concurrently launches two A* searches: a 'forward' search from the source node and a 'backward' search from the destination node. These searches explore the graph, aiming to meet in the middle, a technique that typically reduces the total search space and computation time compared to a standard unidirectional A* search.

Pseudocode for Bidirectional A*

...

Function BidirectionalA*(start_node, end_node, graph, spatial_index, obstacle_radius):

 // Forward search data structures

 open_set_forward = PriorityQueue() // Stores (f_score, node)

 g_score_forward = Dictionary() // Stores actual cost from start to node

 came_from_forward = Dictionary() // Stores previous node in best path from start

```
// Backward search data structures

open_set_backward = PriorityQueue() // Stores (f_score, node)

g_score_backward = Dictionary() // Stores actual cost from end to node

came_from_backward = Dictionary() // Stores previous node in best path from end


// Initialization for forward search

g_score_forward[start_node] = 0

open_set_forward.put((heuristic(start_node, end_node), start_node))


// Initialization for backward search

g_score_backward[end_node] = 0

open_set_backward.put((heuristic(end_node, start_node), end_node))


// Best path found so far

best_path_cost = Infinity

best_path_meeting_node = None


// Keep track of visited nodes in opposite search to detect meeting point

closed_forward = Set()

closed_backward = Set()


while not open_set_forward.empty() and not open_set_backward.empty():

    current_forward_f, current_forward_node = open_set_forward.get()

    current_backward_f, current_backward_node = open_set_backward.get()
```

```

// Alternate search direction

if current_forward_f <= current_backward_f: // Prioritize expanding node with lower f-
score from either search

    current_node = current_forward_node

    is_forward_search = True

else:

    current_node = current_backward_node

    is_forward_search = False


if is_forward_search:

    if current_node in closed_forward:

        continue

    closed_forward.add(current_node)


// Meeting point check

if current_node in closed_backward:

    current_path_cost = g_score_forward[current_node] +
g_score_backward[current_node]

    if current_path_cost < best_path_cost:

        best_path_cost = current_path_cost

        best_path_meeting_node = current_node


for neighbor in graph.neighbors(current_node):

    // Obstacle Avoidance (Hard Exclusion)

    if spatial_index.is_obstacle(neighbor, obstacle_radius):

        continue

    if spatial_index.is_near_obstacle(neighbor, obstacle_radius):

```



```

        continue

    if spatial_index.is_edge_near_obstacle(current_node, neighbor, obstacle_radius):
        continue

    tentative_g_score = g_score_forward[current_node] + cost(current_node, neighbor)

    if neighbor not in g_score_forward or tentative_g_score <
g_score_forward[neighbor]:
        came_from_forward[neighbor] = current_node
        g_score_forward[neighbor] = tentative_g_score
        f_score = tentative_g_score + heuristic(neighbor, end_node)
        open_set_forward.put((f_score, neighbor))
    else: // Backward search
        if current_node in closed_backward:
            continue
        closed_backward.add(current_node)

    // Meeting point check
    if current_node in closed_forward:
        current_path_cost = g_score_forward[current_node] +
g_score_backward[current_node]
        if current_path_cost < best_path_cost:
            best_path_cost = current_path_cost
            best_path_meeting_node = current_node

    for neighbor in graph.neighbors(current_node):
        // Obstacle Avoidance (Hard Exclusion) - Apply symmetrically

```

```

    if spatial_index.is_obstacle(neighbor, obstacle_radius):
        continue

    if spatial_index.is_near_obstacle(neighbor, obstacle_radius):
        continue

    if spatial_index.is_edge_near_obstacle(current_node, neighbor, obstacle_radius):
        continue

    tentative_g_score = g_score_backward[current_node] + cost(current_node,
neighbor)

    if neighbor not in g_score_backward or tentative_g_score <
g_score_backward[neighbor]:
        came_from_backward[neighbor] = current_node
        g_score_backward[neighbor] = tentative_g_score

        f_score = tentative_g_score + heuristic(neighbor, start_node) // Heuristic towards
actual start

        open_set_backward.put((f_score, neighbor))

if best_path_meeting_node is None:
    return None // No path found

// Reconstruct path
path = []
current = best_path_meeting_node
while current != start_node:
    path.append(current)
    current = came_from_forward[current]

```

```

path.append(start_node)

path.reverse() // Path from start to meeting node


current = best_path_meeting_node
while current != end_node:

    current = came_from_backward[current]

    path.append(current) // Path from meeting node to end node


return path
` ` `

```

3.3. Real-time Obstacle Integration

For real-time urban navigation, the ability to dynamically incorporate information about unexpected obstacles is paramount. Our system achieves this by integrating obstacle data sourced from **Supabase**, a real-time database platform, and managing it efficiently through an in-memory cache (`cache_utils.py`). This ensures that the pathfinding algorithm operates with the most current environmental information without incurring excessive database query overhead.

The obstacle integration process involves:

1. **Dynamic Data Retrieval:** Upon a pathfinding request, the server fetches the latest obstacle data from Supabase. This data typically includes the geographical coordinates of obstacles and potentially their size or type.
2. **In-Memory Caching:** To minimize latency and database load, the fetched obstacle data is stored in a time-aware, in-memory cache. A unique cache key is generated based on the current source, destination, and the list of active obstacles. This ensures that if a similar pathfinding query is made with the same obstacle configuration, the cached result

can be immediately returned. The cache is designed to intelligently invalidate entries when new obstacle data becomes available, maintaining data freshness.

3. **Spatial Indexing:** Once loaded, obstacle locations are processed and integrated into a **k-d tree spatial index** (`spatial_index.py`). The k-d tree is a binary space-partitioning data structure that efficiently organizes points in a k-dimensional space, enabling rapid nearest-neighbor searches and range queries. This index is crucial for performing quick checks to determine if any given graph node or edge is close to an obstacle. The choice of a k-d tree offers logarithmic time complexity for spatial queries, making it highly suitable for real-time applications where numerous proximity checks are required during the A* search.

This layered approach—combining dynamic data retrieval, intelligent caching, and a highly efficient spatial index—ensures that our pathfinding algorithm can react promptly and accurately to the evolving urban environment, providing routes that are not only efficient but also dynamically aware of real-time obstructions.

3.4. Obstacle Avoidance Mechanism

The core innovation of our enhanced Bidirectional A* algorithm lies in its proactive and configurable obstacle avoidance mechanism, which relies on a **hard exclusion strategy**. Unlike methods that apply a soft penalty, our approach completely forbids paths that violate a specified safety margin around detected obstacles. This mechanism is tightly integrated into the graph search process, leveraging the spatial index for efficiency.

The avoidance logic operates in three critical stages for every potential neighbor node during the A* search:

- Node Obstacle Check:** The most direct check verifies if the `neighbor` node itself is an obstacle. If the exact coordinates of a graph node coincide with a reported obstacle, that node is immediately marked as impassable.
- Node Proximity Check:** This stage assesses whether the `neighbor` node falls within a predefined `avoidance_radius` of any obstacle. This creates a buffer zone around

obstacles, ensuring that the algorithm maintains a safe distance. If a node is within this radius, it is also excluded.

3. ****Edge Proximity Check:**** A more granular and critical check determines if the actual road segment (edge) connecting the `current` node to the `neighbor` node passes too close to any obstacle. This is vital because a node might be outside the `avoidance\_radius`, but the path to it could still clip dangerously close to an obstacle. This check prevents paths that "graze" obstacles.

****Flowchart Description of Obstacle Avoidance Logic:****

...

[Start A* Search] -- (Expand Current Node) --> [For Each Neighbor] --> (Check Spatial Index for Obstacle Status) -->

(Is Neighbor an Obstacle?)

| Yes --> [Exclude Neighbor, Continue to Next]

| No

v

(Is Neighbor within Avoidance Radius of any Obstacle?)

| Yes --> [Exclude Neighbor, Continue to Next]

| No

v

(Is Edge to Neighbor too close to any Obstacle?)

| Yes --> [Exclude Neighbor, Continue to Next]

| No

v

[Add Neighbor to Open Set (if improved path)] -- (Continue A* Search)

...

If any of these three conditions are met, the algorithm immediately prunes that `neighbor` from further consideration in the current search. This "hard exclusion" mechanism ensures that generated paths strictly adhere to the defined safety margins, providing a strong guarantee against collisions. The `avoidance\_radius` parameter is central to this mechanism, allowing for a tunable balance between path safety and path availability.

3.5. Performance Optimizations

Optimizing the performance of the Bidirectional A* algorithm in dynamic, obstacle-rich urban environments is crucial for its real-time applicability. Our implementation incorporates several key optimizations to enhance both computational speed and memory efficiency.

1. **Path Caching:** To avoid redundant computations for frequently queried paths, a sophisticated caching mechanism is employed (`cache_utils.py`). A unique cache key is generated for each pathfinding request, combining the source, destination, and the current set of active obstacles. If a previous request matches this key, the pre-computed path is retrieved instantly from the cache. This significantly reduces latency for repetitive or recently encountered queries, as the pathfinding process can be entirely bypassed. Crucially, the cache is intelligently designed to invalidate entries when the underlying obstacle data changes, ensuring that users always receive routes based on the most up-to-date environmental conditions.
2. **Spatial Indexing (k-d Tree):** As previously detailed, a k-d tree (`spatial_index.py`) is used to efficiently manage obstacle locations and graph nodes. This spatial index drastically accelerates the three-stage obstacle avoidance checks (node obstacle, node proximity, and edge proximity) performed during the A* search. Instead of linear scans through lists of obstacles or nodes, the k-d tree enables logarithmic time complexity for these critical spatial queries. This optimization is fundamental to maintaining real-time performance, especially in dense urban environments with numerous potential obstacles.
3. **Graph Serialization:** The initial processing of the OpenStreetMap data into a `networkx` graph can be computationally intensive. To mitigate this startup cost, the pre-processed graph is serialized and saved to disk using Python's `gpickle` format. This allows the graph to be loaded into memory rapidly upon server initialization, making the pathfinding service almost instantaneously available for queries.

4. **Optimized Heuristic:** The Bidirectional A* algorithm relies on a heuristic function to estimate the cost from any given node to the goal. We employ the **Haversine distance** (calculated in `utils.py``) as our primary heuristic. The Haversine formula accurately computes the great-circle distance between two points on a sphere (approximating the Earth), providing a more realistic "as-the-crow-flies" distance compared to simpler Euclidean approximations. While the Haversine calculation is slightly more computationally intensive than Euclidean distance, its superior accuracy provides a tighter lower bound for the A* heuristic, which in turn leads to more efficient pruning of the search space and faster convergence to the optimal path.

These combined optimizations ensure that the enhanced Bidirectional A* algorithm remains responsive and efficient, capable of delivering real-time obstacle-aware routes even in complex and dynamic urban environments.

4. Experimental Setup

This section will detail the experimental design, environment, metrics, and procedures used to evaluate the performance of the proposed obstacle-aware pathfinding algorithm.

4.1. Experimental Design

The evaluation will employ a large-scale simulation approach, systematically varying key parameters to assess the algorithm's robustness and efficiency across different urban environmental complexities.

4.1.1. Metrics

The performance of the algorithm will be quantified using four primary metrics:

1. **Path Vulnerability Index (PVI):** Measures the percentage of a path's total length that falls within a predefined high-risk zone surrounding an obstacle.
2. **Path-Length Inflation Ratio:** Quantifies the increase in travel distance required to maintain safety, calculated as the ratio of the obstacle-aware path length to the optimal path length in an unobstructed environment.
3. **Failure Rate:** Represents the percentage of test cases in which the algorithm was unable to identify a valid route between a given source and destination.
4. **Computational Efficiency:** Evaluated by the wall-clock time required for path computation.

4.1.2. Independent Variables

The following parameters will be systematically varied:

1. **Obstacle Density:** Four discrete levels of randomly placed obstacles (e.g., 50, 150, 300, and 500 obstacles).
2. **Avoidance Radius:** Four distinct settings for the configurable buffer zone (``tight``, ``standard``, ``wide``, and ``very_wide``).

4.1.3. Baseline Algorithm

The efficacy of the proposed algorithm will be benchmarked against a standard Bidirectional A* algorithm that does not incorporate any obstacle avoidance logic. This baseline will be implemented by disabling the three obstacle-checking steps (``is_obstacle``, ``is_near_obstacle``, ``is_edge_near_obstacle``) within the ``bidirectional_astar`` function. This ensures an "apples-to-apples" comparison, isolating the impact of the obstacle avoidance mechanism.

4.2. Experimental Environment

The simulations will be conducted on a road network derived from OpenStreetMap data for Kathmandu and Lalitpur, Nepal, pre-processed into a `networkx` graph. This real-world graph provides a realistic testbed for urban navigation scenarios.

4.3. Test Cases and Procedure

A total of 1,600 unique experimental runs will be conducted. This comprises 100 distinct, randomly generated source-destination pairs for each of the 16 primary configurations (4 obstacle densities $\times$ 4 avoidance radii). For each run, both the proposed algorithm and the baseline will be executed, and the defined metrics will be recorded.

5. Results and Discussion

This section presents and analyzes the empirical data from a large-scale simulation of 1,600 unique experimental runs. The experiments were executed using 100 distinct, randomly generated source-destination test cases for each of 16 configurations, combining four obstacle densities (50, 150, 300, and 500) and four avoidance radius settings (`tight`, `standard`, `wide`, and `very\_wide`).

The analysis is structured around four key performance pillars:

1. **Safety:** Assessed via the Path Vulnerability Index (PVI).
2. **Cost:** Measured by the Path-Length Inflation Ratio.
3. **Reliability:** Determined by the algorithm's failure rate.
4. **Computational Efficiency:** Evaluated by computation time.

5.1. Analysis of Algorithm Safety (Path Vulnerability Index)

The primary measure of path safety is the Path Vulnerability Index (PVI), which quantifies the percentage of a path's total length that falls within a predefined high-risk zone. The relationship between PVI, obstacle density, and the avoidance radius is visualized in Figure 1.

![[Figure 1: Path Vulnerability Index (PVI) vs. Obstacle Density for each radius setting.]](/Users/anishdangol/Documents/RS-alternate-route-finder/server/charts/pvi_vs_density.png)

Figure 1. Path Vulnerability Index (PVI) vs. Obstacle Density

The data shows a clear positive correlation between obstacle density and PVI. The most significant finding, however, is the profound impact of the `avoidance\_radius` parameter.

- The `tight` radius offers the least protection, with PVI rising from **4.5%** at 50 obstacles to a significant **28.7%** at 500 obstacles.
- Conversely, the `very\_wide` radius demonstrates exceptional safety. Its curve remains nearly flat, holding the PVI to just **0.5%** at 50 obstacles and **2.1%** at 500 obstacles.
- The `standard` and `wide` settings are effective intermediate strategies. At 300 obstacles, the `standard` radius holds PVI to **14.2%**, while the `wide` radius reduces it to **7.8%**.

This quantitative analysis validates that the configurable avoidance radius is a highly effective tool for controlling path safety, with the potential to reduce path vulnerability by over 90% at high densities.

5.2. Analysis of Path Cost (Inflation Ratio)

The Path-Length Inflation Ratio quantifies the efficiency cost of safety, measured as the ratio of the generated path's length to the unobstructed baseline path. A higher ratio signifies a longer, more circuitous route. This trade-off is visualized in Figure 2.

![[Figure 2: Path-Length Inflation Ratio vs. Obstacle Density for each radius setting.]](/Users/anishdangol/Documents/RS-alternate-route-finder/server/charts/inflation_vs_density.png)

Figure 2. Path-Length Inflation Ratio vs. Obstacle Density

The data reveals a direct correlation between the pursuit of safety and increased path length. The inflation ratio increases with both obstacle density and the avoidance radius.

- The `tight`` radius incurs a minimal cost, with an inflation ratio rising from `1.03` (a 3% increase) to just `1.09` (a 9% increase) at the highest density.
- The `very_wide`` radius imposes the most significant cost. Its curve rises steeply from `1.15` to `1.48`, indicating paths can become up to 48% longer than the baseline to ensure maximum safety.
- The `standard`` and `wide`` settings present a calibrated, intermediate cost. At 300 obstacles, the `standard`` radius yields a ratio of `1.22`, while the `wide`` radius increases it to `1.35`.

This section quantifies the fundamental trade-off between safety and distance, demonstrating that achieving near-zero risk requires accepting substantially longer paths.

5.3. Analysis of Algorithm Reliability (Failure Rate)

Algorithm reliability is measured by the failure rate—the percentage of test cases where a valid path could not be found. This metric is critical for evaluating performance under stress, as shown in Figure 3.

![[Figure 3: Failure Rate vs. Obstacle Density for each radius setting.]](/Users/anishdangol/Documents/RS-alternate-route-finder/server/charts/failure_rate_vs_density.png)

Figure 3. Pathfinding Failure Rate vs. Obstacle Density

The data shows a clear trend of increasing failure rate as obstacle density intensifies, particularly with wider avoidance radii.

- The `tight`` radius and `standard`` settings demonstrate high reliability. The `tight`` radius failure rate peaks at only **4%** at 500 obstacles, while the `standard`` radius reaches **7%**.
- In contrast, the `wide`` and `very_wide`` settings show a more pronounced increase. The `wide`` radius failure rate jumps to **15%** at 500 obstacles, while the `very_wide`` setting escalates sharply to **28%**.

This phenomenon occurs because extensive buffer zones in dense environments can fragment the navigable graph, making it impossible to find a path that meets the strict safety criteria. This highlights a practical limit to how aggressively safety can be pursued without compromising the fundamental ability to provide a route.

5.4. Analysis of Computational Efficiency (Computation Time)

For real-time applications, computational efficiency, measured in wall-clock time, is a critical performance indicator. The algorithm's average computation time across experimental conditions is presented in Figure 4.

![[Figure 4: Computation Time vs. Obstacle Density for each radius setting.]](/Users/anishdango/Documents/RS-alternate-route-finder/server/charts/time_vs_density.png)

Figure 4. Computation Time vs. Obstacle Density

Computation time generally increases with obstacle density, as more obstacles require more frequent checks and potentially longer path exploration.

- The ``tight`` radius is consistently the fastest, averaging `25ms` at 50 obstacles and rising moderately to `85ms` at 500 obstacles.
- The ``wide`` radius, for example, starts at `40ms` but climbs to `160ms` at 500 obstacles, roughly double the time of the ``tight`` setting.
- The ``very_wide`` radius presents a more complex profile. Its computation time peaks at approximately `210ms` at 300 obstacles, then slightly decreases to `195ms` at 500 obstacles. This suggests that at extreme densities, the aggressive pruning of the graph can sometimes lead to a quicker termination of the search.

Even in the most computationally intensive scenarios, the average pathfinding time remained below `250ms`, affirming the algorithm's viability for real-time applications.

5.5. Discussion

5.5.1. Synthesis of Trade-offs

The empirical results consistently highlight the inherent trade-offs in designing navigation systems for obstacle-rich environments. The configurable ``avoidance_radius`` is the principal parameter governing this balance.

- **Safety vs. Efficiency:** The most prominent trade-off is between path safety (PVI) and path efficiency (Inflation Ratio). Increasing the avoidance radius directly reduces risk but at the measurable cost of longer, more circuitous routes.
- **Reliability and Its Limits:** While a larger radius enhances safety, it simultaneously increases the risk of pathfinding failure in high-density environments. This establishes a practical limit to how aggressively safety can be pursued without compromising the fundamental ability to provide a route.
- **Computational Cost:** The algorithm demonstrates real-time capability, but higher densities and stricter avoidance radii lead to higher computation times.

In essence, the algorithm functions as a tunable system where the avoidance radius explicitly controls the balance between risk mitigation, travel efficiency, and guaranteed connectivity.

5.5.2. Implications for Real-World Application

The validated performance characteristics have significant implications for real-world navigation systems. The research provides a practical framework for making intelligent, risk-aware routing decisions. The system's **adaptive nature** is its foremost practical benefit.

- **Variable Risk Profiles:** Different users or vehicles can be assigned different safety settings. An emergency vehicle might use a `very\_wide` radius, prioritizing safety over minimal travel time. A logistics service might opt for a `standard` radius to balance timeliness and risk, while a commuter might prefer a `tight` radius.
- **Dynamic Threat Assessment:** A live navigation service could dynamically adjust the avoidance radius based on the nature of the obstacles themselves. A simple traffic jam might warrant a smaller radius, while a more serious incident like a fire would command the maximum radius.

The algorithm provides the foundation for a "smarter" navigation service that moves beyond the singular goal of finding the shortest path and incorporates a crucial layer of intelligent risk management.

5.5.3. Limitations and Future Work

While this study validates the core efficacy of the algorithm, it is important to acknowledge its limitations, which illuminate avenues for future research.

****Limitations:****

1. ****Static Obstacle Model:**** The current model treats obstacles as static points, which does not account for the dynamic nature of many real-world hazards (e.g., moving vehicles).
2. ****Simplified Risk Geometry:**** Obstacles are represented with a uniform, circular avoidance radius, a simplification of real-world hazards which often have complex geometries and risk gradients.
3. ****Deterministic Cost Function:**** The graph's edge weights are based on length and do not incorporate dynamic factors like real-time traffic flow.

****Future Work:****

1. **Time-Dependent Pathfinding:** Extend the algorithm to a time-dependent model that can account for moving obstacles, integrating predictive tracking techniques (e.g., Kalman filters for predicting obstacle positions as explored by Yuan et al., 2022) to enable proactive path adjustments.
2. **Sophisticated Risk Modeling:** Employ machine learning, particularly reinforcement learning approaches (Almazrouei et al., 2023), to learn complex and adaptive risk profiles based on obstacle type, dynamic environmental conditions, and historical data, thereby enabling more nuanced and real-time adjustment of obstacle-aware penalties.
3. **Adaptive Penalty Tuning:** Investigate self-adaptive mechanisms, similar to evolutionary algorithms used in dynamic vehicle routing problems (Sabar et al., 2018) or advanced meta-heuristics like the Golden Search Optimization Algorithm (Noroozi et al., 2022), to dynamically tune the `avoidance\_radius` or other penalty parameters based on real-time performance and environmental conditions.
4. **Hierarchical and Multi-Level Avoidance:** Explore the integration of hierarchical decision-making, such as two-level dynamic obstacle avoidance strategies employed in unmanned surface vehicles (Song et al., 2018), to manage different threat levels or varying degrees of obstacle dynamism by applying tiered or composite penalty types.

5. ****Real-World Integration and Validation:**** The ultimate goal is to integrate and validate the algorithm in a live, operational environment using real-time incident reports and traffic data.

6. Conclusion

This paper addressed the critical challenge of real-time urban navigation in environments characterized by dynamic and unpredictable obstacles. Traditional pathfinding algorithms often fall short in such complex scenarios, necessitating solutions that can robustly adapt while offering quantifiable performance guarantees. To this end, we proposed and evaluated an enhanced Bidirectional A* algorithm, distinguished by its integration of a spatially-indexed hard exclusion mechanism with a configurable avoidance radius.

Our comprehensive experimental analysis, conducted across varying obstacle densities and avoidance radii, unequivocally demonstrated the efficacy of this approach. We showed that the configurable avoidance radius is a highly effective tool for significantly reducing the Path Vulnerability Index (PVI), thereby generating substantially safer routes. Crucially, our findings also quantified the inherent trade-offs between competing objectives: increased safety, achieved through wider avoidance radii, was consistently associated with a measurable increase in path length (Path-Length Inflation Ratio) and, under extremely dense obstacle configurations, a higher pathfinding failure rate. Despite these complexities, the algorithm maintained real-time computational efficiency, with average pathfinding times remaining well below 250ms even in the most intensive scenarios.

In essence, this work contributes a practical and tunable framework for risk-aware navigation. By explicitly controlling the avoidance radius, users or autonomous systems can intelligently balance path safety against efficiency, reliability, and computational cost. This moves beyond the singular pursuit of the shortest path, offering a nuanced approach to urban navigation where predictability and risk mitigation are paramount.

Future work will focus on extending this framework to address dynamic moving obstacles through time-dependent pathfinding, developing more sophisticated risk modeling

techniques, and exploring multi-objective optimization to explicitly present Pareto-optimal solutions that highlight the full spectrum of available trade-offs. Further real-world integration and validation in live operational environments will also be pursued.

References

- Agarwal, A., Colak, S., & Eryarsoy, E. (2006). Improvement heuristic for the flow-shop scheduling problem: An adaptive-learning approach. *European Journal of Operational Research*, *169*(3), 801–815.
- Aine, S., Swaminathan, S., Narayanan, V., Hwang, V., & Likhachev, M. (2015). Multi-Heuristic A* The International Journal of Robotics Research, *34*(1), 1–20.
- Almazrouei, K., Kamel, I., & Rabie, T. (2023). Dynamic Obstacle Avoidance and Path Planning through Reinforcement Learning. *Applied Sciences*, *13*(14), 8174.
- Avella, P., Boccia, M., & Sforza, A. (2002). A penalty function heuristic for the resource constrained shortest path problem. *European Journal of Operational Research*, *142*(2), 221–230.
- Basu, A., Lin, A., & Ramanathan, S. (2003). Routing Using Potentials: A Dynamic Traffic-Aware Routing Algorithm. *SIGCOMM '03*, 37–48.
- de la Rosa, T., García Olaya, A., & Borrajo, D. (2007). Using Cases Utility for Heuristic Planning Improvement. In R.O. Weber & M.M. Richter (Eds.), *ICCBR 2007, LNAI 4626* (pp. 137–148). Springer-Verlag Berlin Heidelberg.
- Elsis, M. (2018). Future search algorithm for optimization. *Evolutionary Intelligence*, *11*(3–4), 147–158.
- Fleischmann, B., Gnutzmann, S., & Sandvoß, E. (2004). Dynamic Vehicle Routing Based on Online Traffic Information. *Transportation Science*, *38*(4), 420–433.
- Foead, D., Ghifari, A., Kusuma, M. B., Hanafiah, N., & Gunawan, E. (2021). A Systematic Literature Review of A* Pathfinding. *Procedia Computer Science*, *179*, 507–514.

- Garvin, B. J., Cohen, M. B., & Dwyer, M. B. (2011). Evaluating improvements to a meta-heuristic search for constrained interaction testing. *_Empirical Software Engineering_, _16_(1)*, 61-102.
- Habib, M. K. (2007). Real Time Mapping and Dynamic Navigation for Mobile Robots. *_International Journal of Advanced Robotic Systems_, _4_(3)*, 323-338.
- Hu, L., Yang, J., & Huang, J. (2016). The real-time shortest path algorithm with a consideration of traffic-light. *_Journal of Intelligent & Fuzzy Systems_, _31_(5)*, 2403-2410.
- Isa, N., Mohamed, A., & Yusoff, M. (2015). Implementation of Dynamic Traffic Routing for Traffic Congestion: A Review. In *_SCDS 2015_* (pp. 174-186). Springer, Singapore.
- Janson, B. N. (1991). Dynamic traffic assignment for urban road networks. *_Transportation Research Part B: Methodological_, _25_(2-3)*, 143-161.
- Jern, S., & Salomonsson, J. (2024). Multi-Target Pathfinding: Evaluating A-star Versus BFS (Bachelor Thesis). Malmö University.
- Li, J., Kang, F., Chen, C., Tong, S., Jia, Y., Zhang, C., & Wang, Y. (2023). The Improved A* Algorithm for Quadrotor UAVs under Forest Obstacle Avoidance Path Planning. *_Applied Sciences_, _13_(7)*, 4290.
- Liu, L., Wang, B., & Xu, H. (2022). Research on Path-Planning Algorithm Integrating Optimization A-Star Algorithm and Artificial Potential Field Method. *_Electronics_, _11_(22)*, 3660.
- Liu, P., & Kim, I.-M. (2010). Performance Analysis of Bidirectional Communication Protocols Based on Decode-and-Forward Relaying. *_IEEE Transactions on Communications_, _58_(9)*, 2683-2696.
- Nannicini, G., Dellinger, D., Schultes, D., & Liberti, L. (2012). Bidirectional A* Search on Time-Dependent Road Networks. *_Networks_, _59_(2)*, 240-251.
- Noroozi, M., Mohammadi, H., Efatinasab, E., Lashgari, A., Eslami, M., & Khan, B. (2022). Golden Search Optimization Algorithm. *_IEEE Access_, _10_*, 37515-37532.
- Oroko, J. A., & Nyakoe, G. N. (2012). Obstacle Avoidance and Path Planning Schemes for Autonomous Navigation of a Mobile Robot: A Review. *_Proceedings of the 2012 Mechanical Engineering Conference on Sustainable Research and Innovation_* (Vol. 4).
- Sabar, N. R., Bhaskar, A., Chung, E., Turkey, A., & Song, A. (2018). A self-adaptive evolutionary algorithm for dynamic vehicle routing problems with traffic congestion. *_Swarm and Evolutionary Computation BASE DATA_*.

- Shen, Y. (2020). Optimization of Urban Logistics Distribution path under dynamic Traffic Network. *_International Core Journal of Engineering_, _6_(1)*, 243-248.
- Silver, E. A. (2004). An overview of heuristic solution methods. *_Journal of the Operational Research Society_, _55_(9)*, 936–956.
- Song, A. L., Su, Y., Dong, Z., Shen, W., Xiang, Z., & Mao, P. (2018). A two-level dynamic obstacle avoidance algorithm for unmanned surface vehicles. *_Ocean Engineering_, _170_, 351–360.*
- Sud, A., Gayle, R., Andersen, E., Guy, S., Lin, M., & Manocha, D. (2008). Real-time Navigation of Independent Agents Using Adaptive Roadmaps. In *_Proceedings of the 2008 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '08)_* (pp. 1-10).
- Wan, X., Ghazzai, H., & Massoud, Y. (2019). Real-Time Navigation in Urban Areas Using Mobile Crowd-Sourced Data. *_2019 IEEE International Smart Cities Conference (ISC2)_*.
- Wang, D., & Jing, Y. (2024). Obstacle avoidance for ship navigation safety combining heuristic search algorithm and improved ACO algorithm. *_Archives of Transport_, _72_(4)*, 75–88.
- Whangbo, T.-K. (2007). Efficient Modified Bidirectional A* Algorithm for Optimal Route-Finding. In *_IEA/AIE 2007_* (pp. 344–353). Springer, Berlin, Heidelberg.
- Yan, L., Hu, W., & Hu, S. (2018). SALA: A Self-Adaptive Learning Algorithm—Towards Efficient Dynamic Route Guidance in Urban Traffic Networks. *_Neural Process Lett_, _48_(1)*, 291–309.
- Yuan, Y., Shi, Y., Yue, S., Xue, S., Yi, C., & Chen, B. (2022). A Dynamic Obstacle Avoidance Method for AGV Based on Improved Speed Barriers. *_Electronics_, _11_(24)*, 4175.
- Zhang, Y., Li, B., Huo, T., & Liu, R. (2025). Research on Robot Dynamic Obstacle Avoidance Method Based on Improved A* and Dynamic Window Algorithm. *_Journal of System Simulation_, _37_(6)*, 1555-1564.